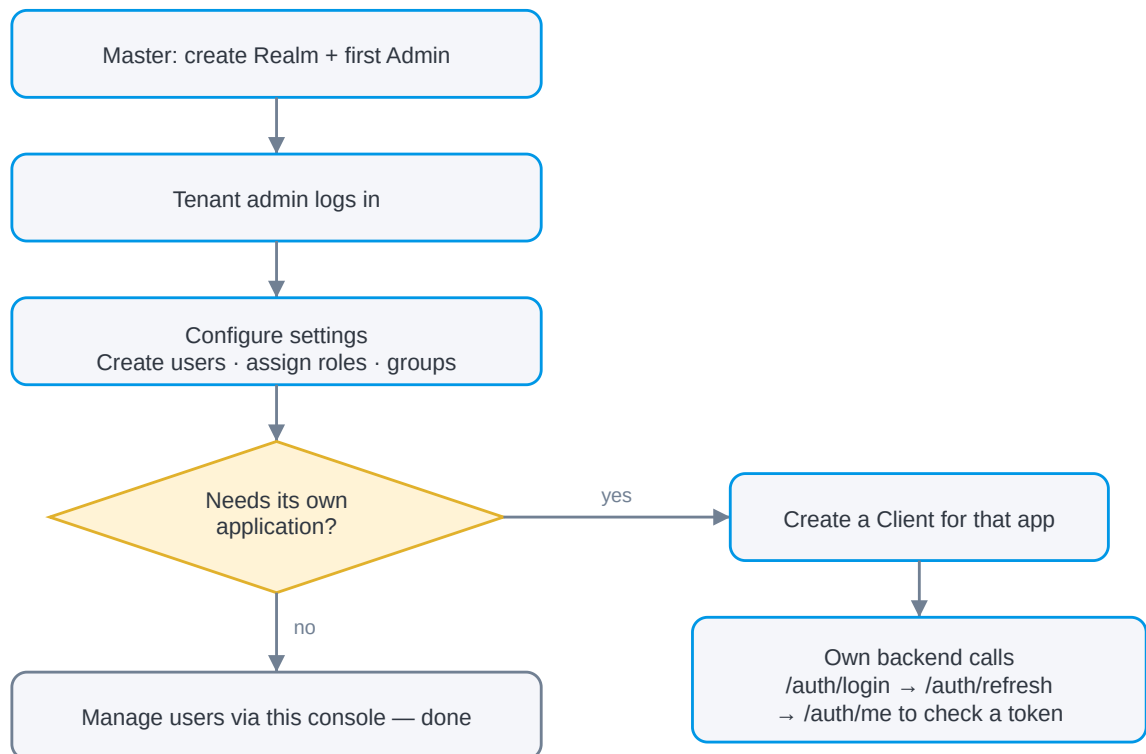


New Tenant Onboarding Guide

Setting up a new tenant realm on the IAM admin console — main steps, master admin and tenant admin, plus integrating your own backend.



Part 1 — Master admin: create the realm

1 Create the realm and its first admin user in one step.

Go to [Realms → Create Realm](#). Provide the realm name and the details for the tenant's first admin (username, email, password). This single action: creates the realm, links it to the shared admin-console client with a fresh access code, seeds default realm settings, and assigns that first user the ADMIN role plus an "Administrators" group.

2 Hand off the login details to the tenant.

They need three things to log in: the client's access code (crAccessCode) for their new realm, and the admin's email + password you just set.

A tenant never creates its own realm — that's a Master-only action by design. Everything from here on is done by the tenant's own admin, once they're logged in.

Part 2 — Tenant admin: first login and setup

3 Log in.

Enter the access code from step 2 along with the admin email and password. No realm picker is needed — the access code identifies the realm automatically.

4 Review realm settings.

Go to [Settings → Advanced settings](#) and check:

Setting	Why it matters
---------	----------------

Allow registration	Turn on if end users should be able to self-sign-up instead of being created manually
Email SMTP	Required for verify-email and reset-password emails to actually send
Access / refresh token TTL	Defaults to 15 minutes / 7 days — adjust if the tenant needs shorter or longer sessions

- 5 **Create additional users.**
[Users → Create](#) for anyone who isn't self-registering. New users get no role by default — assign one next.
- 6 **Assign roles.**
 From a role's [Users](#) tab (or a user's [Roles](#) tab), assign ADMIN / MANAGER / VIEWER / AUDITOR / USER as appropriate. These roles come from the shared admin-console client — the tenant doesn't need to create its own.
- 7 **Group users (optional).**
[Groups → Create](#) to organize users and assign roles at the group level instead of one by one — useful once the user count grows.
- 8 **Create the tenant's own application client (only if they have one).**
 If the tenant is building a separate app that needs its own OAuth-style login (not just admins using this console), go to [Clients → Create](#) to get a client ID/secret, redirect URIs, and that app's own resources/permissions/roles. Not needed if the tenant only manages users through this console.
- 9 **Spot-check the setup.**
 Open [Sessions](#), [Refresh Tokens](#), and [Events](#) to confirm logins are being recorded, and note that any session or token can be revoked from those pages if something looks wrong.

Part 3 — Using this as the identity provider for your own backend

Only relevant if the tenant is building its own separate application and wants this system to handle its login/users instead of building that themselves.

- 10 **Create a Client for that application.**
[Clients → Create](#). Choose *Confidential* if the app has a secure backend that can hold a secret, or *Public* for a client-side-only app. This also generates a fresh access code for this client + this tenant's realm.
- 11 **Log a user in from your backend.**
`POST /auth/login` with header `x-cr-access-code: <the code from step 10>` and body `{ email, password, clientSecret? }` (clientSecret only for Confidential clients). Returns `{ accessToken, refreshToken, user }`. The access token is a short-lived (1 hour) JWT; the refresh token lasts 7 days by default.
- 12 **Validate a token on every protected request.**
 Access tokens are signed with a secret this API keeps to itself — there's no public key or introspection endpoint, so a tenant backend can't verify the signature on its own. Call `GET /auth/me` with `Authorization: Bearer <accessToken>`; a 200 response with the user object means the token is valid.
- 13 **Refresh and log out.**
 When the access token expires, `POST /auth/refresh` with `{ refreshToken }` returns a new access token (and rotates the refresh token). `POST /auth/logout` ends the session and revokes its tokens.

In short: treat this API as the single source of truth for auth — the tenant's backend proxies login/refresh calls to it and calls `/auth/me` to check a token, rather than trying to verify tokens itself.